

FUNCTIONS IN PYTHON

Definition: Functions are the subprograms that perform specific task. Functions are the small modules.

Types of Functions:

There are **two** types of functions in python:

1. Library Functions (Built in functions)
2. Functions defined in modules
3. User Defined Functions

1. Library Functions in Python:

These functions are already built in the library of python.

For example: `type( )`, `len( )`, `input( )` etc.

2. Functions defined in modules:

a. Functions of math module:

To work with the functions of math module, we must import math module in program.

import math

S. No.	Function	Description	Example
1	<code>sqrt()</code>	Returns the square root of a number	<pre>>>>math.sqrt(49) 7.0</pre>
2	<code>ceil()</code>	Returns the upper integer	<pre>>>>math.ceil(81.3) 82</pre>
3	<code>floor()</code>	Returns the lower integer	<pre>>>>math.floor(81.3) 81</pre>
4	<code>pow()</code>	Calculate the power of a number	<pre>>>>math.pow(2,3) 8.0</pre>
5	<code>fabs()</code>	Returns the absolute value of a number	<pre>>>>math.fabs(-5.6) 5.6</pre>
6	<code>exp()</code>	Returns the e raised to the power i.e. e^3	<pre>>>>math.exp(3) 20.085536923187668</pre>

b. Function in random module:

random module has a function `randint( )`.

- `randint( )` function generates the random integer values including start and end values.
- Syntax: `randint(start, end)`
- It has two parameters. Both parameters must have integer values.

Example:

```
import random  
n=random.randint(3,7)
```

*The value of n will be between 3 and 7.

View all functions available in a module: `dir(module_name)`

Eg. `dir(random)`

3. USER DEFINED FUNCTIONS:

The syntax to define a function is:

def function-name (parameters) :

#statement(s)

Where:

- Keyword **def** marks the start of function header.
- A function name to uniquely identify it. Function naming follows the same rules of writing identifiers in Python.
- Parameters (arguments) through which we pass values to a function. They are optional.
- A colon (:) to mark the end of function header.
- One or more valid python statements that make up the function body. Statements must have same indentation level.
- An optional return statement to return a value from the function.

Example:

```
def display(name):  
    print("Hello " + name + " How are you?")  
display("JECRC")
```

Function Parameters:

A functions has two types of parameters:

1. **Formal Parameter:** Formal parameters are written in the function header of the definition. Formal parameters are local variables which are assigned values from the arguments when the function is called.

2. Actual Parameter: When a function is *called*, the values that are passed in the call are called *actual parameters*. At the time of the call each actual parameter is assigned to the corresponding formal parameter in the function definition.

Example:

```
def ADD(x, y):                #Defining a function and x and y are formal parameters
    z=x+y
    print("Sum = ", z)
    a=float(input("Enter first number: " ))
    b=float(input("Enter second number: " ))
ADD(a,b)                     #Calling the function by passing actual parameters
```

In the above example, **x** and **y** are formal parameters. **a** and **b** are actual parameters.

Calling the function:

Once we have defined a function, we can call it from another function, program or even the Python prompt. To call a function we simply type the function name with appropriate parameters.

Syntax:

function-name(parameter)

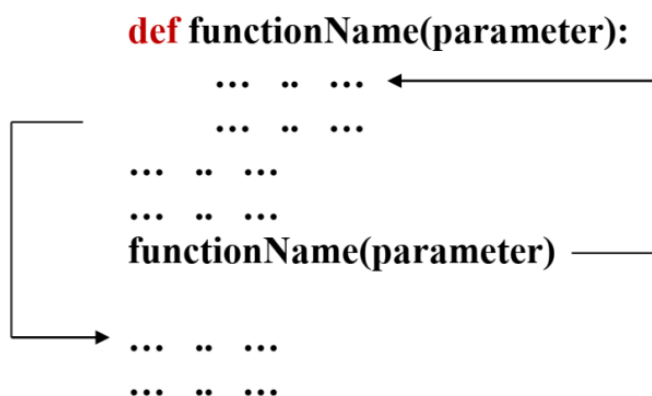
Example:

ADD(10,20)

OUTPUT:

Sum = 30.0

How function works?



The return statement:

The **return** statement is used to exit a function and go back to the place from where it was called.

There are two types of functions according to return statement:

- a. Function returning some value (non-void function)
- b. Function not returning any value (void function)

a. Function returning some value (non-void function) :

Syntax:

return expression/value

Example-1: Function returning one value

```
def my_function(x):  
    return 5 * x
```

Example-2 Function returning multiple values:

```
def sum(a,b,c):  
    return a+5, b+4, c+7  
  
S=sum(2,3,4)      # S will store the returned values as a tuple  
print(S)
```

OUTPUT: (7, 7, 11)

Example-3: Storing the returned values separately:

```
def sum(a,b,c):  
    return a+5, b+4, c+7  
  
s1, s2, s3=sum(2, 3, 4)      # storing the values  
separately print(s1, s2, s3)
```

OUTPUT:

7 7 11

b. Function not returning any value (void function): The function that performs some operations but does not return any value, called void function.

```
def message():  
    print("Hello")
```

```
m=message()  
print(m)
```

OUTPUT: Hello

None

5.5 Scope and Lifetime of variables:

Scope of a variable is the portion of a program where the variable is recognized. Parameters and variables defined inside a function is not visible from outside. Hence, they have a local scope.

There are two types of scope for variables:

1. Local Scope
2. Global Scope

1. **Local Scope:** Variable used inside the function. It can not be accessed outside the function. In this scope, The lifetime of variables inside a function is as long as the function executes. They are destroyed once we return from the function. Hence, a function does not remember the value of a variable from its previous calls.

2. **Global Scope:** Variable can be accessed outside the function. In this scope, Lifetime of a variable is the period throughout which the variable exists in the memory.

Example:

```
def my_func():  
    x = 10  
    print("Value inside function:",x)  
  
x = 20  
my_func()  
print("Value outside function:",x)
```

OUTPUT:

Value inside function: 10

Value outside function: 20

Here, we can see that the value of **x** is **20** initially. Even though the function `my_func()` changed the value of **x** to **10**, it did not affect the value outside the function.

This is because the variable **x** inside the function is different (local to the function) from the one outside. Although they have same names, they are two different variables with different scope.

On the other hand, variables outside of the function are visible from inside. They have a **global scope**.

We can read these values from inside the function but cannot change (write) them. In order to modify the value of variables outside the function, they must be declared as global variables using the keyword **global**.

RECURSION:

Definition: A function calls itself, is called recursion.

5.6.1 Python program to find the **factorial of a number** using recursion:

Program:

```
def factorial(n):  
    if n == 1:  
        return n  
    else:  
        return n*factorial(n-1)  
  
num=int(input("enter the number: "))  
  
if num < 0:  
    print("Sorry, factorial does not exist for negative numbers")  
elif num == 0:  
    print("The factorial of 0 is 1")  
else:  
    print("The factorial of ",num," is ", factorial(num))
```

OUTPUT:

enter the number: 5

The factorial of 5 is 120

Python program to print the Fibonacci series using recursion: Program:

```
def fibonacci(n):  
    if n<=1:  
        return n  
    else:  
        return(fibonacci(n-1)+fibonacci(n-2))  
  
num=int(input("How many terms you want to display: "))  
  
for i in range(num):  
    print(fibonacci(i)," ", end=" ")
```

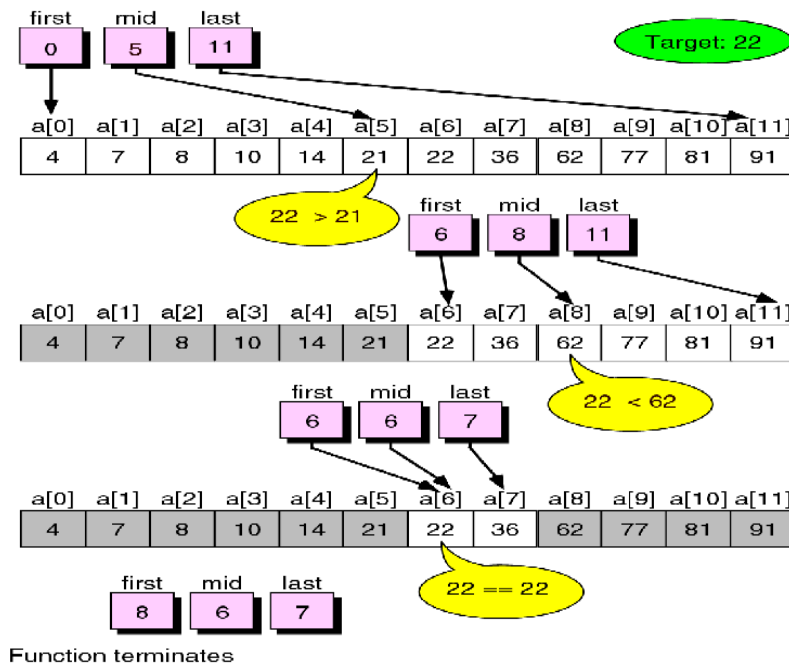
OUTPUT:

How many terms you want to display: 8

0 1 1 2 3 5 8 13

Binary Search using recursion:

Note: The given array or sequence must be sorted to perform binary search.



Program:

```
def Binary_Search(sequence, item, LB, UB):  
    if LB>UB:
```

```

        return -5          # return any negative
    value mid=int((LB+UB)/2)
    if item==sequence[mid]:
        return mid
    elif item<sequence[mid]:
        UB=mid-1
        return Binary_Search(sequence, item, LB, UB)
    else:
        LB=mid+1
        return Binary_Search(sequence, item, LB, UB)

L=eval(input("Enter the elements in sorted order: "))
n=len(L)
element=int(input("Enter the element that you want to search :"))
found=Binary_Search(L,element,0,n-1)
if found>=0:
    print(element, "Found at the index : ",found)
else:
    print("Element not present in the list")

```

lambda Function:

lambda keyword, is used to create anonymous function which doesn't have any name.

While normal functions are defined using the **def** keyword, in Python anonymous functions are defined using the **lambda** keyword.

Syntax:

lambda arguments: expression

Lambda functions can have any number of arguments but only one expression. The expression is evaluated and returned. Lambda functions can be used wherever function objects are required.

Example:


```
value = lambda x: x * 4  
print(value(6))
```

Output:
24

In the above program, `lambda x: x * 4` is the lambda function. Here `x` is the argument and `x * 4` is the expression that gets evaluated and returned.